

Chapter 7: Applications of AI

14 hours | 26 marks — THE MOST IMPORTANT CHAPTER

"In theory, theory and practice are the same. In practice, they are not."

— Albert Einstein (attributed)

The Hook — Where It All Comes Together

We've built up:

- A framework for problems (Ch.2)
- Search algorithms (Ch.3)
- Logic and reasoning (Ch.4)
- Knowledge structures (Ch.5)
- Learning principles (Ch.6)

Now: **what do we actually BUILD with all of this?**

This chapter covers the three flagship applications of AI:

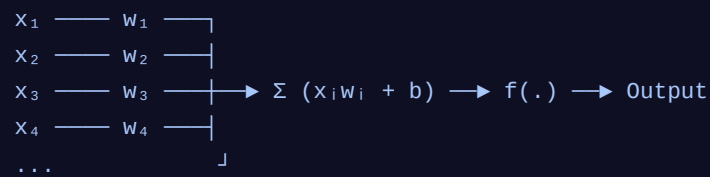
1. **Neural Networks** — the workhorse of modern AI
2. **Expert Systems** — AI capturing human expertise
3. **NLP and Machine Vision** — AI that sees and understands language

These 26 marks demand deep understanding. Let's go layer by layer.

PART 1: Neural Networks

7.1 Network Structure — From Biology to Math

The Artificial Neuron (Perceptron)



where:

x_i = input signals
 w_i = weights (how important is each input)
 b = bias (threshold shifter)
 $f(.)$ = activation function

The math:

$$\text{net} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \sum_i w_ix_i + b$$

$$\text{output} = f(\text{net})$$

Activation Functions — The Non-Linear Magic

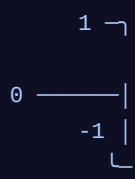
Without non-linear activation functions, a multi-layer network collapses into a single linear operation. Non-linearity is what gives networks their power.

Function	Formula	Graph Shape	Used Where
Step	$f(x) = 1 \text{ if } x \geq \theta, \text{ else } 0$	Square step	Original perceptron
Sigmoid	$f(x) = 1/(1+e^{-x})$	S-curve	Binary classification (output)
Tanh	$f(x) = (e^x - e^{-x})/(e^x + e^{-x})$	S-curve centered at 0	Hidden layers
ReLU	$f(x) = \max(0, x)$	Hockey stick	Modern deep nets (hidden)
Softmax	$f(x_i) = e^{x_i} / \sum e^{x_j}$	Probability distribution	Multi-class output

Sigmoid (σ):



Tanh:



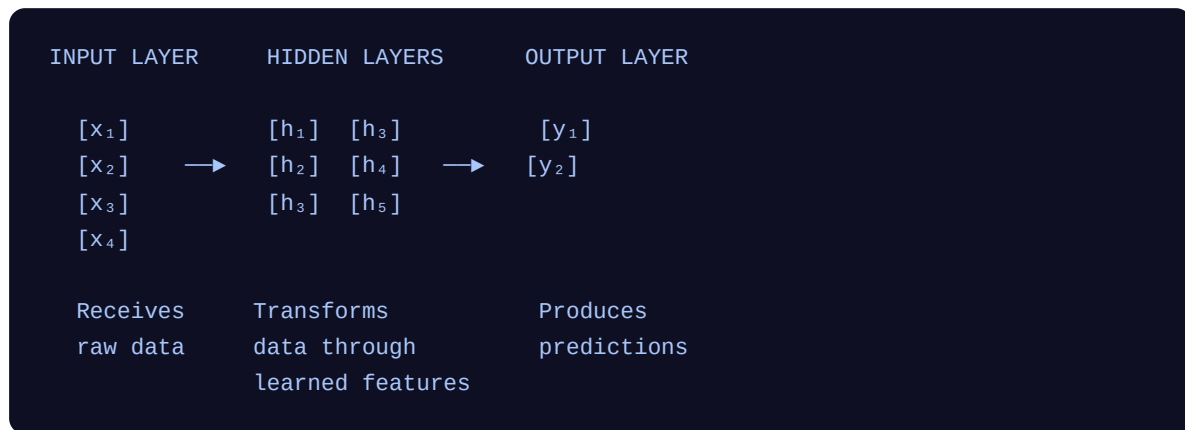
ReLU:



Intuition for sigmoid: Squashes any input to (0,1). Perfect for expressing probability.

Intuition for ReLU: "Fire only if positive." Simple, fast, doesn't cause vanishing gradients as severely.

Network Layers

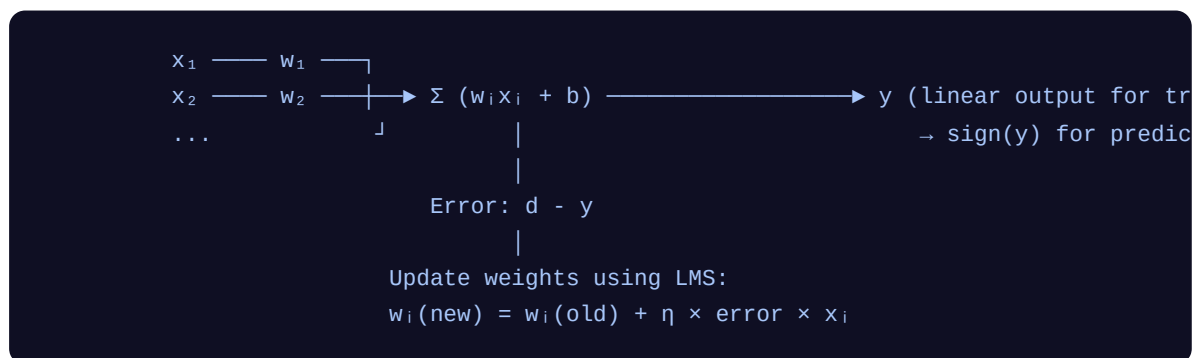


7.2 Adaline Network

What Is Adaline?

Adaline = Adaptive Linear Neuron (developed by Widrow and Hoff, 1960)

It's a single-layer network that uses **continuous** (not stepped) output during training.



The Widrow-Hoff Learning Rule (LMS — Least Mean Squares)

$$w_i \leftarrow w_i + \eta \times (d - \hat{y}) \times x_i$$

where:

η = learning rate (how big a step to take)

d = desired output

$\hat{y}$ = actual output
 x_i = input that produced this output

Intuition: If my output $\hat{y}$ is too low (error positive), increase weights that contributed to this output.

Adaline vs. Perceptron

Feature	Perceptron	Adaline
Update rule	Only when misclassified	Every step (continuous error)
Output during training	Binary (0/1)	Continuous (linear)
Convergence	Converges if data linearly separable	Converges to minimum MSE
Loss function	0/1 loss	Squared error (MSE)

7.3 The Perceptron

The Original Neural Network

Frank Rosenblatt's **Perceptron** (1958) was the first learning algorithm for neural networks.

PERCEPTRON LEARNING ALGORITHM:

1. Initialize weights randomly (or to 0)
2. For each training example (x , desired_output d):
 - a. Compute output: $\hat{y} = f(\sum w_i x_i + b)$
 - b. If $\hat{y} = d$: No update (correct!)
 - c. If $\hat{y} \neq d$:
 - $w_i \leftarrow w_i + \eta \times (d - \hat{y}) \times x_i$
 - $b \leftarrow b + \eta \times (d - \hat{y})$
3. Repeat until all examples classified correctly
 OR maximum iterations reached

The Perceptron Convergence Theorem

If the training data is linearly separable, the perceptron WILL converge to a correct solution in a finite number of steps.

What is linearly separable?

Linearly separable: NOT linearly separable:
(XOR problem)

```

  x  x
o    o
o    o    x  x
  _____
One line separates them

  o  x
  x  o
  
```

No straight line works!

The perceptron's fatal flaw (Minsky & Papert, 1969):

It cannot solve XOR. This caused the first AI winter.

7.4 Multilayer Perceptron & Backpropagation

The Solution to XOR: Add Hidden Layers

```

INPUT LAYER      HIDDEN LAYER      OUTPUT LAYER
x1 ————— h1 ————— y
                \
x2 ————— h2
  
```

With hidden units, the network can learn non-linear boundaries!
XOR: Hidden layer transforms the input space so it becomes linearly separable.

Backpropagation — The Algorithm That Changed Everything

The Problem: How do we know what each weight in the middle layers contributed to the final error?

The Answer: Chain rule of calculus — propagate the error BACKWARD through the network.

FORWARD PASS (CALCULATE OUTPUT)

Given input x , compute output through all layers:

```

z1 = W1 · x + b1      (layer 1 linear transformation)
a1 = f(z1)            (layer 1 activation)
z2 = W2 · a1 + b2    (layer 2 linear transformation)
a2 = f(z2)            (layer 2 activation)
ŷ = a2                 (final output)
  
```

Loss: $L = (1/2)(\hat{y} - y)^2$ (mean squared error)

BACKWARD PASS (UPDATE WEIGHTS)

Using the chain rule to find $\partial L / \partial w_{ij}$ for each weight:

```
Output layer error:
δ_output = (ŷ - y) × f'(z_output)

Hidden layer error (backpropagated):
δ_hidden = (W_output^T · δ_output) × f'(z_hidden)

Weight update:
ΔW = -η × δ × a^T
W ← W + ΔW
```

Intuitive explanation (3Blue1Brown style):

Imagine tuning a stereo with many knobs. You hear the music is too bassy. You don't know exactly which knob causes it. Backpropagation tells you: *"the bass knob in the 3rd row accounts for 40% of the bass problem — turn it down a bit."* Each weight gets its "blame" calculated.

```
FORWARD:  x → Layer1 → Layer2 → Output → Error
BACKWARD: x ← ∂L/∂w₁ ← ∂L/∂w₂ ← ∂L/∂w₃ ← ∂L/∂ŷ
```

THE CHAIN RULE AT THE CORE

$$\partial L / \partial w_1 = \partial L / \partial \hat{y} \times \partial \hat{y} / \partial a_2 \times \partial a_2 / \partial z_1 \times \partial z_1 / \partial w_1$$

Each term is computed locally — this makes the algorithm efficient!

COMPLETE BACKPROP WORKED EXAMPLE (XOR)

Training XOR with hidden layer [2 inputs → 2 hidden → 1 output]:

Training sample: input=(0,1), target=1

Step 1 — Forward pass:

```
w_h = [[0.5, 0.6], [0.3, 0.8]] (weights to hidden)
w_o = [0.7, 0.5]                (weights to output)

h_input = [0×0.5+1×0.3, 0×0.6+1×0.8] = [0.3, 0.8]
```

```

h_output = [sigmoid(0.3), sigmoid(0.8)] = [0.574, 0.690]

o_input = 0.574×0.7 + 0.690×0.5 = 0.402 + 0.345 = 0.747
o_output = sigmoid(0.747) = 0.678

Error = 0.678 - 1 = -0.322
Loss = 0.5 × 0.3222 = 0.052

```

Step 2 — Backward pass:

```

δ_output = -0.322 × sigmoid'(0.747)
           = -0.322 × 0.678(1-0.678) = -0.322 × 0.218 = -0.070

δ_h1 = δ_output × w_o[0] × sigmoid'(0.3)
       = -0.070 × 0.7 × 0.574(1-0.574) = -0.012

δ_h2 = δ_output × w_o[1] × sigmoid'(0.8)
       = -0.070 × 0.5 × 0.690(1-0.690) = -0.0075

Weight updates (η=0.5):
w_o[0] += η × δ_output × h_output[0] = 0.5 × (-0.070) × 0.574 → w_o[0] decreases
w_o[1] += η × δ_output × h_output[1] = 0.5 × (-0.070) × 0.690 → w_o[1] decreases

```

After thousands of iterations, the network learns XOR!

7.5 Hopfield Network

What Is It?

A **Hopfield Network** (1982) is a **recurrent** neural network — connections go in both directions! It functions as an **associative memory**.



All neurons connected bidirectionally.
 $w_{ij} = w_{ji}$ (symmetric weights)

The Energy Function

Like Boltzmann Machines, Hopfield networks have an energy:

$$E = -\frac{1}{2} \sum_{i,j} w_{ij} s_i s_j + \sum_i \theta_i s_i$$

where:

s_i = state of neuron i (+1 or -1)

w_{ij} = weight of connection between i and j

θ_i = threshold of neuron i

Key property: The network always evolves to **lower energy states**.

Stored patterns = **local minima** of the energy function.

How Hopfield Networks Store Memories

Training (Hebbian Learning):

For each pattern p to store:

$w_{ij} += p_i \times p_j$ (increase weight if both neurons agree)

$w_{ii} = 0$ (no self-connections)

Retrieval (Energy minimization):

1. Start with a **partial or noisy** version of a stored pattern
2. Update neurons: $s_i \leftarrow \text{sign}(\sum_j w_{ij} s_j - \theta_i)$
3. Repeat until stable (energy minimum reached)
4. The network settles at the stored pattern closest to the input!

RETRIEVAL EXAMPLE:

Stored pattern: [+1 +1 -1 +1 -1]
(think of this as "the letter A")

Input (noisy): [+1 -1 -1 +1 -1]
(some bits corrupted)

After settling: [+1 +1 -1 +1 -1] ← Retrieved original! ✓

This is exactly how human memory works — we can recall full memories from partial cues! See someone's face and remember their name.

Capacity Limit

A Hopfield network with N neurons can store approximately **$0.14N$ patterns** before patterns start corrupting each other.

7.6 Kohonen Self-Organizing Map (SOM)

What Is It?

A **Kohonen SOM** (Teuvo Kohonen, 1982) is an **unsupervised** neural network that creates a **2D map** of high-dimensional data.

The Big Idea: Organize neurons so that similar inputs activate neurons that are physically close together — just like the brain's cortex!

HIGH-DIMENSIONAL INPUT SPACE:
(house: 5-room, 300m², suburb, 15yo, ...)

↓

KOHONEN MAP (2D grid):

expensive	moderate
large	medium
cheap	moderate
small	medium

Similar houses cluster together on the map!

How It Works (Training)

For each training example x :

1. **COMPETITION:** Find the winning neuron (BMU):

$$\text{BMU} = \underset{j}{\operatorname{argmin}} ||x - w_j|| \quad (\text{closest weight vector})$$
2. **COOPERATION:** Define a neighborhood around BMU

$$h(t) = \exp(-\text{dist}^2 / 2\sigma(t)^2) \quad \leftarrow \text{Gaussian neighborhood}$$

$$(\sigma \text{ decreases over time} - \text{neighborhood shrinks})$$
3. **ADAPTATION:** Update BMU and neighbors:

$$w_j(t+1) = w_j(t) + \eta(t) \times h(t) \times (x - w_j(t))$$

(pull weight vectors toward the input)

Intuition: If input x is "expensive large house", the closest neuron moves toward it. Its neighbors also move a bit, but less. The neighborhood shrinks over time.

Applications

Application	What the SOM Organizes
Data visualization	Project high-dim data to 2D for human viewing
Word embeddings	Similar words cluster together
Medical diagnosis	Similar patient profiles cluster
Market segmentation	Similar customer profiles cluster

PART 2: Expert Systems

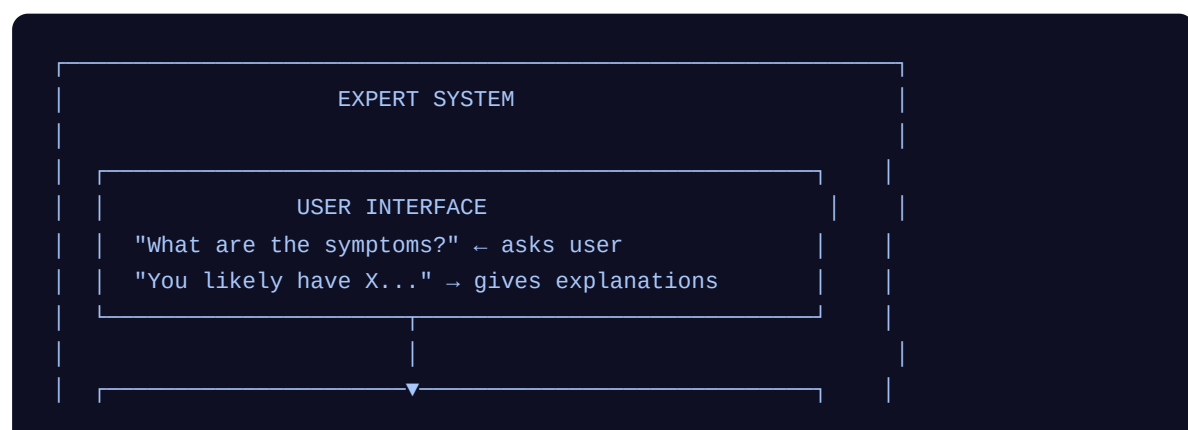
7.7 Architecture of an Expert System

What Is an Expert System?

An expert system is an AI program that mimics the decision-making ability of a human expert in a specific domain.

Classic example: MYCIN (1970s) — diagnosed bacterial infections and recommended antibiotics. In tests, it outperformed most doctors (except top specialists)!

The Architecture





7.8 Knowledge Acquisition and Induction

The Bottleneck

"The knowledge acquisition bottleneck" — the hardest part of building an expert system is getting knowledge OUT of human experts' heads and INTO the system.

Experts often can't articulate what they know! ("I just know it's the right answer.")

Methods of Knowledge Acquisition

Method	How	Pros	Cons
Interviews	Ask expert structured questions	Direct	Expert can't always explain
Observation	Watch expert work	Captures tacit knowledge	Time-consuming
Case studies	Analyze past decisions	Real-world data	Might not generalize

Method	How	Pros	Cons
Protocol analysis	Expert thinks aloud while working	Captures reasoning process	Artificial situation
Machine induction	Learn rules from example cases	Automatic	Needs many labeled examples

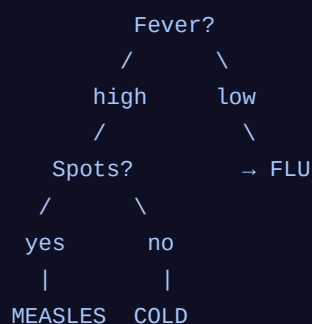
Knowledge Induction (Machine Approach)

Instead of interviewing experts, show the system **cases with outcomes**:

Cases:

```
(fever=high, cough=yes, runny-nose=yes) → COLD
(fever=high, cough=no, spots=yes) → MEASLES
(fever=low, fatigue=high, cough=no) → FLU
```

ID3 Algorithm builds a decision tree:



The system has **induced** medical knowledge from cases — no expert interview needed!

7.9 Knowledge Representation in Expert Systems

Declarative Knowledge

Facts that are directly stated:

```
% Facts about drugs
dangerous_drug(penicillin) :- patient_allergic_to(penicillin).
effective_for(amoxicillin, strep_throat).
requires_prescription(amoxicillin).
```

Procedural Knowledge

How to do something step-by-step:

```
PROCEDURE: Diagnose-Bacterial-Infection
STEP 1: Get patient symptoms
STEP 2: Check fever → IF fever > 38.5°C, suspect bacterial infection
STEP 3: Check WBC count → IF WBC > 11000, confirm bacterial infection
STEP 4: Identify type → culture test
STEP 5: Select antibiotic based on type and patient allergies
```

Certainty Factors (Handling Uncertainty)

MYCIN used certainty factors (CF) from -1 to +1:

- CF = 1.0: Certainly true
- CF = 0.0: Unknown
- CF = -1.0: Certainly false

Combining certainty factors:

If both evidence E1 (CF=0.7) and E2 (CF=0.6) support H:

$$CF(H) = CF(E1) + CF(E2) \times (1 - CF(E1)) = 0.7 + 0.6 \times 0.3 = 0.88$$

7.10 Development of Expert Systems

The MYCIN Story (Case Study in Depth)

Background:

- Stanford, early 1970s
- Goal: diagnose bacterial blood infections
- Development: 5+ years, team of doctors + AI researchers

Knowledge base:

- ~450 production rules like:

```
IF: organism is gram-negative AND
    patient is compromised host AND
```

```
infection site is blood
THEN: organism is Pseudomonas (CF=0.6)
```

Performance:

- Tested against 10 Stanford physicians
- MYCIN performance: 65% correct recommendations
- Average physician: 42.5%
- Only infectious disease specialist: 62.5%

Lessons learned:

- Domain-specific systems can outperform general physicians
- Knowledge acquisition is the real bottleneck
- Systems need explanation facilities (doctors won't trust black boxes)
- Expertise is surprisingly brittle at domain boundaries

PART 3: NLP and Machine Vision

7.11 Natural Language Processing (NLP)

The Levels of Language Analysis

Human language has multiple layers. NLP works at each level:

```
RAW TEXT: "The student who passed the exam was happy"
|
▼ PHONETIC/PHONOLOGICAL ANALYSIS
Sound patterns: /ðə 'stju:dənt hu: pɑ:st ðə ɪg'zæm wəz 'hæpi/
(Mapping sounds to phonemes)
|
▼ MORPHOLOGICAL ANALYSIS
Word structure: student (study+ent), passed (pass+ed), happy (free morpheme)
|
▼ SYNTACTIC ANALYSIS (PARSING)
Grammar structure: [NP [Det The] [N student] [RC who passed the exam]] [VP was happy]
|
▼ SEMANTIC ANALYSIS
Meaning: Subject=student, Predicate=happy, Condition=passing-exam
```

|

▼ PRAGMATIC ANALYSIS

Context/Intent: Possibly teacher describing test results, or student's relief

Level 1: Phonetic Analysis

Deals with: Sounds of language

Key concepts:

- **Phoneme:** Smallest unit of sound (/p/, /b/, /k/)
- **Phonology:** Rules about sound combinations
- **Task:** Speech recognition — converting audio to text

Waveform → Phonemes → Words
 ~~~~ /h/ /ə/ /l/ /ou/ → "hello"

**Applications:** Siri, Google Assistant, voice commands

## Level 2: Syntactic Analysis (Parsing)

**Deals with:** Grammatical structure of sentences

Key tools:

**Context-Free Grammar (CFG):**

```

S → NP VP
NP → Det N | Det N PP | ...
VP → V NP | V NP PP | ...
Det → "the" | "a"
N → "student" | "book" | ...
V → "read" | "wrote" | ...
PP → Prep NP

```

**Parse Tree for "The student read a book":**

```

      S
     / \
    NP  VP
   / \ / \

```

```

Det  N  V   NP
|    |  |   /  \
"the" "student" "read" Det  N
                        |    |
                        "a"  "book"

```

### Parsing algorithms:

- **CKY Algorithm:** Bottom-up, dynamic programming
- **Earley Parser:** Handles any CFG
- **Shift-Reduce:** LR parsing

## Level 3: Semantic Analysis

**Deals with:** Meaning of words and sentences

Challenges:

- **Word sense disambiguation:** "bank" = financial institution OR riverbank?
- **Anaphora resolution:** "John hit Bill. He fell." — who fell?
- **Compositionality:** Meaning of sentence  $\neq$  just sum of word meanings

### Word Embeddings (Modern approach):

```

"king" - "man" + "woman"  $\approx$  "queen"
(famous Word2Vec result – words have geometric relationships in semantic space!)

```

```

Vector space:
  King •
    ↑ "royalty" dimension
    |
  Man • —————> Woman • —————> Queen •

```

## Level 4: Pragmatic Analysis

**Deals with:** Language in context — what the speaker means vs. what they say

Key concepts:

- **Speech Acts:** "Can you pass the salt?" isn't a question — it's a request



- **Implicature:** "Some students passed" implies (pragmatically) not all did
- **Discourse structure:** How sentences relate across a document

## 7.12 Introduction to Machine Vision

### What Is Machine Vision?

Machine vision enables computers to **interpret** and **understand** images and video from the real world.

```
CAMERA → PREPROCESSING → FEATURE → INTERPRETATION
captures raw noise removal extraction classification
pixel data edge detection texture detection
normalization shape recognition
```

### The Fundamental Tasks

| Task                    | Description                          | Example                     |
|-------------------------|--------------------------------------|-----------------------------|
| <b>Classification</b>   | What is in this image?               | "This is a cat"             |
| <b>Detection</b>        | Where are objects in the image?      | Bounding boxes around faces |
| <b>Segmentation</b>     | Which pixels belong to which object? | Pixel-perfect car outline   |
| <b>Recognition</b>      | Who is this person/object?           | Face recognition            |
| <b>Depth estimation</b> | How far away is this?                | Self-driving car distance   |

### The Pipeline (Classical Approach)

```
Step 1: IMAGE ACQUISITION
Camera → Raw pixels → Digital image (array of pixel values)

Step 2: PREPROCESSING
Grayscale conversion, noise filtering, contrast adjustment

Step 3: EDGE DETECTION
Find boundaries where pixel values change sharply
(Canny edge detector, Sobel operator)
```

```

Step 4: SEGMENTATION
  Group pixels into regions (blobs, superpixels)

Step 5: FEATURE EXTRACTION
  Describe regions: SIFT, HOG, color histograms

Step 6: CLASSIFICATION
  Map features to category labels (SVM, neural network)

```

## Convolutional Neural Networks (CNNs) — The Deep Learning Revolution

Modern machine vision uses CNNs:

```

INPUT IMAGE          CONV LAYERS          FULLY CONNECTED
[32x32 pixels]
  |
  ▼
[FILTER 1: edge] → feature map 1 →
[FILTER 2: curve] → feature map 2 → POOLING → FLATTEN → [Dense] → CAT 0.95
[FILTER 3: color] → feature map 3 →                                DOG 0.04
                                                                BIRD 0.01

```

**The key insight:** Convolutional filters automatically LEARN what features to extract from data!

Early layers learn: edges, corners

Middle layers learn: textures, patterns

Later layers learn: object parts, whole objects

## Math Intuition Corner

### The Backpropagation Chain Rule — Visually

```

Loss L depends on output ŷ
ŷ depends on z (pre-activation)
z depends on w (weights)

∂L/∂w = ∂L/∂ŷ × ∂ŷ/∂z × ∂z/∂w

Each term:
  ∂L/∂ŷ: How does loss change when output changes?
  ∂ŷ/∂z: Derivative of activation function

```

$\partial z / \partial w$ : This is just  $x$  (the input to this layer)!

So:

$$\begin{aligned}\partial L / \partial w &= \text{error\_signal} \times f'(z) \times x \\ &= \delta \times x\end{aligned}$$

This is gradient descent: move weights in the direction that reduces the loss.

## Gradient Descent

$$w \leftarrow w - \eta \times \partial L / \partial w$$

Visualization:



$$\text{Step size} = \eta \times \text{gradient}$$



## Video Resources

| Topic                  | Video                                                       | Why Watch                      |
|------------------------|-------------------------------------------------------------|--------------------------------|
| Neural Networks intro  | <a href="#">But what is a neural network? — 3Blue1Brown</a> | The best neural net intro ever |
| Backpropagation visual | <a href="#">Backpropagation — 3Blue1Brown</a>               | Pure visual intuition          |
| Backprop calculus      | <a href="#">Backprop calculus — 3Blue1Brown</a>             | The math behind it             |
| Expert Systems         | <a href="#">Expert Systems intro</a>                        | Architecture walkthrough       |
| NLP Levels             | <a href="#">NLP overview — Stanford</a>                     | Academic depth                 |
| CNN for Vision         | <a href="#">Convolutional Neural Networks — 3Blue1Brown</a> | Visual magic                   |

| Topic             | Video                                           | Why Watch             |
|-------------------|-------------------------------------------------|-----------------------|
| Word Embeddings   | <a href="#">Word2Vec explained — StatQuest</a>  | Intuitive explanation |
| Hopfield Networks | <a href="#">Hopfield Networks — Lex Fridman</a> | From an MIT professor |

## Worked Examples

### Example 1: Trace Forward/Backward Pass

**Network:** 2 inputs → 2 hidden (sigmoid) → 1 output (sigmoid)

**Input:**  $x = [1, 0]$ , target  $y = 1$

**Weights:**  $W1 = [[0.5, 0.1], [0.3, 0.2]]$ ,  $b1 = [0, 0]$ ,  $W2 = [0.4, 0.6]$ ,  $b2 = 0$

**Forward pass:**

```

z1 = W1·x + b1 = [0.5×1 + 0.1×0, 0.3×1 + 0.2×0] = [0.5, 0.3]
a1 = sigmoid([0.5, 0.3]) = [0.622, 0.574]

z2 = W2·a1 + b2 = 0.4×0.622 + 0.6×0.574 = 0.249 + 0.344 = 0.593
ŷ = sigmoid(0.593) = 0.644

Loss = 0.5 × (0.644 - 1)2 = 0.5 × 0.127 = 0.063

```

**Backward pass:**

```

δ_output = (0.644 - 1) × sigmoid'(0.593)
           = -0.356 × 0.644×(1-0.644) = -0.356 × 0.229 = -0.0816

ΔW2[0] = η × δ_output × a1[0] = 0.1 × (-0.0816) × 0.622 = -0.00508
ΔW2[1] = η × δ_output × a1[1] = 0.1 × (-0.0816) × 0.574 = -0.00468

δ_h1 = δ_output × W2[0] × sigmoid'(z1[0])
       = -0.0816 × 0.4 × 0.622×(1-0.622) = -0.0816 × 0.4 × 0.235 = -0.00767

```

### Example 2: Expert System Trace

**KB Rules (for fever diagnosis):**

```

R1: IF temperature > 38.5 AND symptoms contains 'cough' THEN condition = 'flu' (CF=0.7)
R2: IF temperature > 39.5 THEN severity = 'high' (CF=0.9)
R3: IF condition = 'flu' AND severity = 'high' THEN recommend = 'hospitalize' (CF=0.8)

```

**Working memory (user input):** {temperature=39.8, symptoms=['cough', 'headache']}

**Forward chaining:**

1. R1 fires: temperature=39.8>38.5 ✓, 'cough' in symptoms ✓ → condition='flu' (CF=0.7)
2. R2 fires: temperature=39.8>39.5 ✓ → severity='high' (CF=0.9)
3. R3 fires: condition='flu' ✓, severity='high' ✓ → recommend='hospitalize' (CF=0.504)

Combined CF of R3's conclusion =  $0.7 \times 0.9 \times 0.8 = 0.504$

**Output:** "Recommend hospitalization (CF=0.50)"

### Example 3: Parse Tree

**Sentence:** "The AI system learns from data"

**Grammar:**

```

S → NP VP
NP → Det Adj N | Det N
VP → V PP | V Adv
PP → Prep NP

```

**Parse tree:**

```

      S
     / \
    NP  VP
   /|\  | \
  Det Adj N V PP
   |  |  | | / \
 "The" "AI" "system" "learns" Prep NP
                                |  | \
                                "from" Det N
                                    |  |
                                    "ø" "data"

```

## ⚡ Exam-Ready Summary

### Neural Network Architecture Summary

| Network     | Type         | Key Feature               | Use Case                  |
|-------------|--------------|---------------------------|---------------------------|
| Perceptron  | Single-layer | Linear classifier         | Binary classification     |
| Adaline     | Single-layer | Continuous learning       | Regression                |
| MLP         | Multi-layer  | Hidden layers, non-linear | Complex classification    |
| Hopfield    | Recurrent    | Energy-based memory       | Associative memory        |
| Kohonen SOM | Unsupervised | Self-organizing map       | Clustering, visualization |

### Backpropagation Steps

1. Forward pass: compute outputs
2. Compute loss:  $L = (\hat{y} - y)^2/2$
3. Backward pass: compute  $\delta = \partial L / \partial z$  using chain rule
4. Update weights:  $w \leftarrow w - \eta \times \partial L / \partial w$

### Expert System Components

1. **Knowledge Base** — IF-THEN rules + facts
2. **Inference Engine** — applies rules (forward/backward chaining)
3. **Working Memory** — current facts
4. **User Interface** — interaction with user
5. **Explanation Facility** — WHY/HOW questions

### NLP Levels of Analysis

| Level         | Analyzes          | Example                   |
|---------------|-------------------|---------------------------|
| Phonetic      | Sounds            | /hɛləʊ/ = "hello"         |
| Morphological | Word structure    | "running" = "run" + "ing" |
| Syntactic     | Grammar structure | Parse tree                |

| Level     | Analyzes       | Example                   |
|-----------|----------------|---------------------------|
| Semantic  | Meaning        | Word sense disambiguation |
| Pragmatic | Context/intent | Speech acts               |

## Machine Vision Pipeline

Image → Preprocessing → Edge detection → Segmentation → Feature extraction → Classif

## ✓ Chapter 7 Checklist

### Neural Networks:

- ☐ Draw a neuron with all components labeled
- ☐ List and explain 4 activation functions
- ☐ Explain why XOR requires multiple layers
- ☐ Trace forward pass for simple MLP
- ☐ Explain backpropagation in your own words (intuition + math)
- ☐ Explain what a Hopfield network does and how it stores memories
- ☐ Explain what a Kohonen SOM does with a real application
- ☐ Explain Adaline's weight update rule

### Expert Systems:

- ☐ Draw the complete expert system architecture
- ☐ Explain the knowledge acquisition bottleneck
- ☐ Distinguish declarative from procedural knowledge
- ☐ Trace a forward-chaining inference with 3 rules
- ☐ Name 5 real-world expert systems and their domains

### NLP and Vision:

- ☐ Name and describe all 4 levels of NLP analysis
- ☐ Draw a parse tree for a given sentence
- ☐ Explain what word embeddings are

- ☐ List 3 tasks in machine vision
- ☐ Explain the CNN pipeline briefly

---

## Navigation

← **Previous:** [Chapter 6 — Machine Learning](#)

→ **Next:** *(Completed! Review using exam strategy below)*

 **Home:** [README](#)

---

## Final Exam Strategy

### The 80-Mark Battle Plan

```
Priority 1 (26 marks): Chapter 7
  → Master backpropagation mathematically
  → Know ALL 5 neural network types
  → Expert system architecture – draw from memory
  → Know all 4 NLP levels with examples

Priority 2 (14 marks): Chapter 4
  → Truth tables for all connectives
  → Resolution refutation – trace step by step
  → Bayes theorem calculation practice
  → FOPL translation exercises

Priority 3 (9+10 marks): Chapters 3 and 6
  → Search algorithm properties table
  → A* trace with  $f=g+h$ 
  → Alpha-beta pruning
  → GA crossover and mutation
  → Fuzzy membership calculation

Priority 4 (7+7+7 marks): Chapters 1, 2, 5
  → Definitions are key here
  → Semantic net drawings
  → Frame structures
  → State space formulation
```

### Exam Time Management

If exam is 3 hours, 80 marks:



- ~2.25 minutes per mark
- 26-mark chapter: ~1 hour
- 14-mark chapter: ~30 minutes
- Others: distribute remaining time

**Answer Chapter 7 questions first — they're worth the most and you've studied them most!**